

Implementation Plan (Revised v2)

Architecture Summary

- 11 participants: 1 orchestrator (deterministic) + 1 environment (LLM) + 5 firms (LLM) + 4 financial agents (LLM)
- Each agent is a standalone application with own LLM, SQLite database, analytical tools, and multi-step reasoning pipeline (3+ LLM calls per turn)
- Each agent can use a different LLM backend (local Ollama, OpenRouter, Anthropic API, OpenAI API, etc.) configured independently
- Orchestrator is pure Python: accounting, validation, turn control, regime application, dashboard UI
- Communication: HTTP + JSON between orchestrator and agents
- Two configurable regimes: information (what agents see) and measurement (how statements are built)
- Failure mechanics: firms default (with non-automatic entry), banks can become distressed, equity market reprices, M&A with goodwill
- Simulation modes: public or private start, with optional IPO path; complexity toggles (doc 14)
- 4 financial agents: Equity Market (pricing/subscription, PE or public mode) + Investment Bank (advisory) + Commercial Bank + Credit Fund
- Default config: single product (SRT), single world market, most complexity toggles OFF
- Memory: separate within-run (filtered by info regime) and across-run (full info, dispatched post-run) per agent
- Scoring: financial returns for all players, environment rated 1-10 by all agents, cross-run policy database
- Industry Gazette: quarterly trade-publication summary saved and distributed
- Dashboard: Streamlit inspection UI for live monitoring, single-run drill-down, and cross-run comparison
- Compustat panel: ~76 columns, matching real Compustat Quarterly coverage
- N firms and N quarters: user-configurable at start
- World is user-configurable: world docs are input parameters, not hardcoded
- Deployment: zip-per-agent, one machine for dev, distributed for production

Requirements Traceability

#	Requirement	Addressed In
1	Separate memory folders: (a) withi	Doc 09, Phase 5
2	End-of-run memory dispatch + user	Doc 09, Phase 5
3	Non-automatic entry, max N firms,	Doc 10, Phase 2
4	Scoring: financial returns + envir	Doc 11, Phase 8
5	Industry Gazette each period, save	Doc 09, Phase 5+6
6	Agents self-summarize; never uploa	Doc 09, Phase 3+5
7	M&A with financing, goodwill, impa	Doc 10, Phase 2+6
8	User can change the world (world d	Phase 0 (config), Phase 6 (prompt
9	World includes accounting rules do	World doc 08, Phase 6
10	Compustat completeness (~76 column	Doc 12, Phase 1
11	N firms and N quarters configurabl	Phase 0 (config)
12	Orchestrator dashboard UI: time-se	Doc 13, Phase 9
13	Each agent configurable with own A	Phase 7

Diagnosed Gaps (additional items not in user's 1-13)

Gap	Description	Resolution	Status
A. Tax loss carryforward	NOL carryforward affects	World doc 08 (accounting	CLOSED
B. Diluted shares	Options/warrants from sto	World doc 09 (stock comp	CLOSED
C. Currency effects	Global market but single-	USD-denominated. Noted as	ACCEPTED SIMPLIFICATION
D. Lease accounting	Firms lease facilities vs	World doc 09 (lease vs. b	CLOSED
E. Employee stock compens	Part of SGA in biotech, n	World doc 09 (stock comp	CLOSED
F. Dividend policy constr	Negative RE firms paying	Orchestrator blocks divid	CLOSED
G. Antitrust in M&A	Environment blocks merger	Doc 10: max_combined_mark	CLOSED
H. Cross-run DB migration	Schema changes between ve	Version field in run_summ	CLOSED (implementation in
I. Agent observability	Debugging decisions	Full reasoning trace in m	CLOSED
J. Determinism	Reproducibility	Seed propagation: run_see	CLOSED
K. Graceful shutdown	Mid-run interruption	Checkpoint after every qu	CLOSED

Additional Gaps Identified in Consistency Review

Gap	Description	Resolution	Status
L. Capacity utilization i	COGS didn't include utili	Turn protocol Phase 6 upd	CLOSED
M. AE demand modifier mis	Doc 02 defined modifiers	Market demand doc updated	CLOSED
N. Numeric parameters und	Weights, thresholds, rate	World doc 09 (Parameters	CLOSED
O. Workforce effects unde	Hiring/layoff decisions h	Doc 09 defines: R&D speed	CLOSED
P. IPO denial scenario	What happens if investmen	Firm does not enter. Slot	CLOSED
Q. Fair value of R&D in M	Goodwill computation need	Rule: Expensed R&D has \$0	CLOSED
R. Gen 1 COGS inconsisten	Different numbers in diff	Resolved: \$14,000 is the	CLOSED

Phased Build Order

Phase 0: Project Scaffold and Configuration

Goal: Clean structure, config system, world doc loading.

Key decisions:

- World docs are INPUT PARAMETERS stored in config/worlds/default/. The user can create alternate worlds (e.g., config/worlds/electric_vehicles/) with different docs and the simulation runs in that world.
- N firms (initial + max) and N quarters are in config.yaml.
- Each agent's LLM backend is independently configured.

Deliverables:

- [] Project directory structure (see below)
- [] pyproject.toml with dependencies
- [] config/default.yaml -- full configuration with all parameters
- [] config/worlds/default/ -- the SRT world docs (01-08)
- [] config/regimes/ -- information and measurement regime presets
- [] shared/ package: schemas, data types, config loader, world doc loader
- [] Config validation (n_firms, n_quarters, regimes, LLM backends)
- [] CLI skeleton: run, smoke, inspect, dashboard, memory

Agent LLM configuration:

```
# Default: all agents use DeepSeek V3.2 via OpenRouter (~$2.50 per 80Q run)
default_llm:
  backend: "openrouter"
  model: "deepseek/deepseek-v3.2"
  api_key_env: "OPENROUTER_API_KEY"
  temperature: 0.0

# Per-agent overrides (optional -- any agent can use a different model)
agents:
  env_0:
    temperature: 0.2    # environment benefits from slight creativity

  # Example: use a budget model for firms during testing
  # firm_0:
  #   model: "mistralai/mistral-nemo"    # $0.06/M tokens, ~$0.25/run

  # Example: use a local model for a specific agent
  # firm_1:
  #   backend: "ollama"
  #   model: "llama3.2:3b"
  #   ollama_host: "http://localhost:11434"
```

World customization:

```
world:
  docs_dir: "config/worlds/default"    # or "config/worlds/my_custom_world"
  # User can swap in entirely different world docs for different industries
```

Phase 1: Accounting Core (Expanded Compustat)

Goal: All ~76 Compustat columns produced correctly from economic inputs.

Deliverables:

- [] Data types: FirmState (expanded with intanq, gdwlg, aociq, loq, etc.)
- [] Data types: QuarterFlows (expanded with workforce, provisions, M&A, restructuring)
- [] Posting logic: journal entries for all transaction types including:
 - Standard operations (revenue, COGS, SGA, R&D, depreciation, interest, taxes)
 - R&D capitalization and amortization (measurement regime dependent)
 - Fair value adjustments (measurement regime dependent)
 - M&A: goodwill creation, impairment testing
 - Restructuring charges
 - Tax loss carryforward (NOL tracking)
 - Working capital dynamics (AR aging, bad debt, inventory build)
 - Provisions (litigation reserves, warranty)
- [] Statement formatters: IS (18 lines), BS (35 lines), CF (9 lines)
- [] Measurement regime application (6 presets)
- [] Compustat row builder (~76 columns)
- [] Derived ratios: EPS, book value per share, dividends per share
- [] Validation: all hard invariants (expanded set from doc 12)

- [] Unit tests for all posting types

Test: `pytest tests/orchestrator/test_accounting.py -- 50+ test cases covering all transaction types, all measurement regimes, all invariants.`

Phase 2: Orchestrator Mechanics

Goal: Full turn loop with hardcoded inputs. Entry/exit/M&A logic.

Deliverables:

- [] Turn engine: 9-phase quarter loop
- [] Feasibility clamping (priority-order)
- [] Settlement: cash reconciliation, revolver draw, solvency check
- [] Bankruptcy: stochastic-recovery auction, creditor waterfall, slot vacancy
- [] Entry logic: environment-decided entry (not automatic), slot management,

death-spiral tracker, min/max firm count enforcement

- [] M&A mechanics: proposal validation, financing feasibility check, accounting (goodwill creation), integration effects, slot vacancy creation

- [] Goodwill impairment testing (every 4 quarters)
- [] Institution capital: tracking, distress thresholds, constraints, replacement
- [] Information filter: filter full state by observer type + info regime
- [] Shock generation: seeded RNG for macro, taste, recovery, events
- [] Multinomial logit fallback demand model
- [] Checkpoint/resume: serialize full state after each quarter

Test: Orchestrator runs 20 quarters with hardcoded inputs. Include scenarios: firm default in Q5, entry in Q7, M&A in Q12, bank stress in Q15. All invariants pass. Goodwill created on M&A, impaired later. Bank capital declines on default.

Phase 3: Agent Application Core

Goal: Agent application runs standalone, with mock LLM and full analysis stack.

Deliverables:

- [] FastAPI server: `/turn`, `/sync`, `/archive`, `/reset`, `/health`, `/memory/summary`
- [] Brain: 6-step reasoning pipeline (observe-analyze-reflect-strategize-decide-store)
- [] Memory: SQLite with tables (decisions, observations, reflections, analyses, `gazette_notes`)
- [] Context manager: short-term (full detail, last K quarters), medium-term (summarized),

long-term (compressed narrative). Agents self-summarize when context grows.

Never upload raw data to LLM -- run analysis first, include results.

- [] Analyst toolkit:
 - `trend_analysis()`, `summary_statistics()`, `correlation_analysis()`
 - `margin_analysis()`, `cash_runway()`, `rd_efficiency()` (firms)
 - `portfolio_exposure()`, `credit_metrics()`, `historical_default_rates()` (institutions)
 - `valuation_dcf()`, `comparable_firms()` (investment bank)
 - `demand_model_check()`, `event_probability_check()` (environment)
- [] Cross-run retrieval: query past Compustat panels for similar firm-quarters
- [] Mock LLM: returns role-appropriate fixed JSON for all phases

- [] Gazette reader: stores gazette, generates interpretation + action items

Test: Start agent server, POST turn contexts of increasing complexity. Memory.db populates correctly. Analysis tools return correct results. Context manager produces prompts under token budget.

Phase 4: Integration -- Mock Full Run

Goal: Orchestrator + agents run complete simulation with mock LLMs.

Deliverables:

- [] Orchestrator starts/connects to 9 agent servers
- [] IPO sub-sequence (firm requests, ibank prices, banks offer credit)
- [] Full quarter loop, all 9 phases, all agents called correctly
- [] Information regime filtering verified (agents see only allowed data)
- [] Measurement regime verified (statements differ under different regimes)
- [] Firm dossiers created at IPO, updated each quarter by environment
- [] Industry Gazette produced each quarter, distributed to all agents
- [] Industry ledger maintained
- [] Compustat panel: ~76 columns, all invariants pass
- [] Checkpoint/resume works
- [] Memory dispatch tested (simulate run-end, verify archives)
- [] Scoring computed (financial returns + placeholder environment ratings)

Test: `python -m llm_firm_lab smoke --mock --quarters 20 --seed 42` Panel validated. Dossiers populated. Gazettes saved. Checkpoint files created. Scenario includes: 1 default, 1 entry, 1 mock M&A.

Phase 5: Memory System and Cross-Run Learning

Goal: Full memory architecture with within-run, across-run, and dispatch.

Deliverables:

- [] Within-run memory:
 - Per-agent SQLite (decisions, observations, reflections, analyses, gazette notes)
 - Environment keeps full state + per-player archives
 - Filtered by information regime (agents store only what they saw)
- [] Memory context builder:
 - Short-term (last K=4 quarters): full detail
 - Medium-term (K+1 to 3K): agent self-summarizes (LLM call)
 - Long-term (>3K): compressed rolling narrative
 - Cross-run: top-K similar cases from past simulations
 - Token budget enforcement (max_prompt_memory_tokens)
- [] Industry Gazette: generated by environment, saved to disk, distributed to agents,

agents store with interpretation

- [] End-of-run dispatch: orchestrator sends full unfiltered data to all agents
 - Agents archive in `across_runs/run_{id}/`
 - Agents generate `run_summary.md` (LLM call)
 - Agents update `policy_lessons.md` (LLM call)

- Agents update historical_stats.db (SQL aggregation)
- [] User truncation CLI: `python -m llm_firm_lab memory truncate --keep-last 5`
- [] Memory status CLI: `python -m llm_firm_lab memory status`

Test: Run 10 quarters. Memory.db has correct entries. Self-summarization produces reasonable output. End-of-run dispatch populates across_runs/. Truncation works.

Phase 6: Prompt Engineering

Goal: High-quality prompts for all agent types, all phases.

Deliverables:

- [] World doc loader: reads from configured world directory (user-customizable)
- [] System prompts for all agent types:
 - Include relevant world docs (accounting rules, product science, etc.)
 - Include fingerprint/personality
 - Include behavioral constraints and output format
- [] Turn prompts for all phases + all agent types:
 - IPO: request, pricing, initial credit (3 institution types)
 - Quarterly: firm decisions (expanded: workforce, working capital, provisions, M&A)
 - Market resolution: environment (with dossier updates + gazette generation)
 - Financial quarterly: equity pricing (with DCF framework), revolver, term debt
 - Entry decision: environment evaluates whether entry is plausible
 - M&A: proposal, counter-offer, regulatory review, financing
 - Rate environment: end-of-run (all agents rate environment 1-10)
 - Debrief: end-of-run reflection
- [] Reasoning-step prompts (analysis, reflection, strategy, decision)
- [] Valuation prompts (DCF framework, scenario projection, cross-check)
- [] Dossier update prompts (environment: update firm dossiers + industry ledger)
- [] Gazette prompt (environment: write trade-publication quarterly summary)
- [] JSON schemas for all outputs (firm decisions, environment outcomes,

institution terms, ratings, M&A proposals)

- [] Schema validation + re-prompt logic (2 retries, then fallback)

Test: Generate prompts for 5 sample quarters across different scenarios. Manual review. JSON schemas validate against mock outputs. All prompts under context window limits.

Phase 7: LLM Integration (Multi-Backend)

Goal: Each agent independently calls its configured LLM.

Deliverables:

- [] LLM backend interface (abstract base class)
- [] Ollama backend: HTTP client with retry, timeout, seed
- [] OpenRouter backend: HTTP client for any model via openrouter.ai
- [] Anthropic API backend: direct Anthropic SDK
- [] OpenAI-compatible backend: any OpenAI-compatible API

- [] Mock backend: deterministic for testing
- [] Backend factory: instantiate from config (agent-level LLM config)
- [] JSON repair loop: extract first JSON, re-prompt on failure, 2 retries
- [] Fallback behavior on LLM failure (per agent type)
- [] Token counting: estimate prompt size, warn if near limit
- [] Agent-level configuration: each of the 9 agents can use a different

backend/model/temperature independently

Test: Single-quarter run with at least 2 different backends (e.g., Ollama for firms, mock for institutions). All agents produce valid JSON. Reasoning traces are coherent.

Phase 8: Full Runs + Scoring + Diagnostics

Goal: Complete 20+ quarter runs. Score all participants. Diagnose problems.

Deliverables:

- [] Scoring system:
 - Firm: equity IRR, multiple, total shareholder return, operational metrics
 - Institutions: debt IRR, loss rate, risk management metrics
 - Investment bank: pricing RMSE, MAPE, bias, correlation
 - Environment rating: all agents rate 1-10 on 6 dimensions post-run
- [] Cross-run policy database:
 - data/scores.csv: per-actor scores + strategy metrics + regime info
 - data/run_summaries.csv: per-run aggregate metrics
 - Policy context builder for prompts (what strategies worked in past runs?)
- [] Diagnostics CLI:
 - Hard invariant verification
 - Dispersion metrics (are firms differentiating?)
 - Degeneracy detection (identical actions, zero investment, etc.)
 - Death-spiral detection
 - Bank capital trajectory
 - Pricing quality breakdown
 - M&A summary
- [] Run outputs:
 - Compustat panel (local + appended to data/)
 - Debrief CSV (local + appended to data/)
 - Scores CSV (appended to data/)
 - Run summary CSV (appended to data/)
 - Quarterly statements, dossiers, gazettes
 - Agent memory archives (for post-analysis)
 - Orchestrator log

Test: `python -m llm_firm_lab run --quarters 20 --seed 1` completes. Scoring produces reasonable numbers. Environment ratings collected. Cross-run DB populated. Diagnostics CLI runs without errors.

Phase 9: Dashboard UI

Goal: Streamlit dashboard for inspection and visualization.

Deliverables:

- [] Live run monitor: key metrics updated each quarter
- [] Single-run time-series plots:
 - Revenue, market share, profitability (per firm)
 - Balance sheet composition (stacked bars)
 - Cash flow waterfall
 - R&D spending and technology generation
 - Equity prices and pricing errors
 - Institution capital and portfolio
 - Market dynamics (prices, demand, HHI)
- [] Single-quarter drill-down:
 - Full financial statements
 - Firm decisions (requested vs. clamped)
 - Reasoning traces
 - Dossier snapshot
 - Gazette
- [] Cross-run comparison:
 - Side-by-side line charts (same metric, different runs)
 - Regime comparison tables
 - Strategy-outcome scatter plots
- [] All-runs aggregate:
 - Database statistics
 - Strategy-outcome heatmaps (fingerprint style x outcome)
 - Environment quality tracking across runs
 - Regime comparison (aggregated)
- [] CLI inspection (non-GUI):
 - `python -m llm_firm_lab inspect --run-id X --firm firm_0 --quarter 14`
 - `python -m llm_firm_lab plot --run-id X --type revenue --output plots/`

Test: Dashboard launches, loads sample data, all pages render. Plots export. CLI inspection produces correct output.

Phase 10: Regime Experiments

Goal: Systematic runs under different regimes and seeds.

Deliverables:

- [] Experiment runner: config file specifying regime x seed combinations
 - [] Batch execution with progress tracking
 - [] Cross-experiment comparison in dashboard
 - [] Summary report generation (LaTeX or HTML)
-

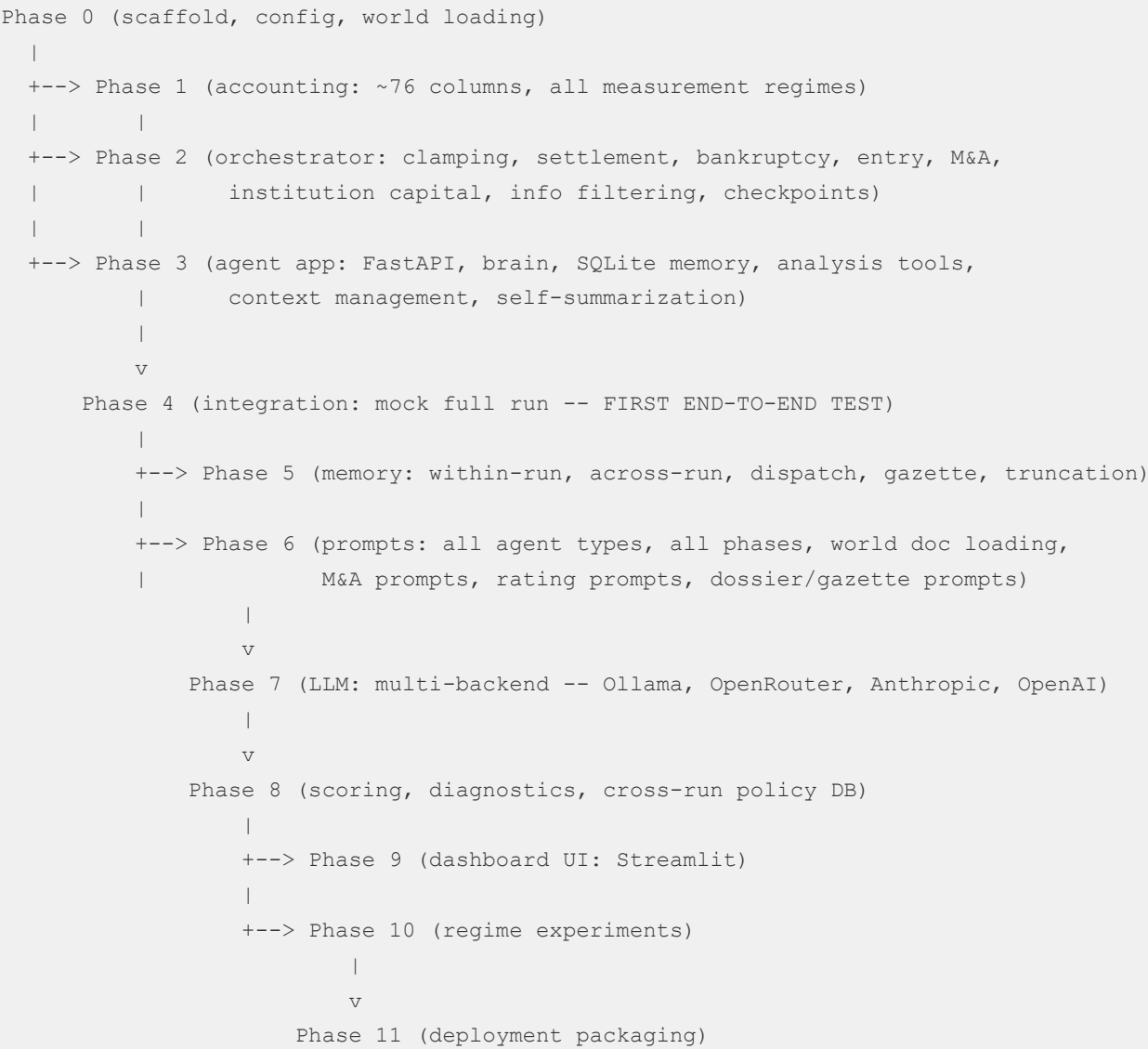
Phase 11: Deployment Packaging

Goal: Zip packages for distributed deployment.

Deliverables:

- [] build_agent_pack.py: assembles zip per agent type
 - Includes: agent source, prompts, shared data, requirements, setup script
 - Config template with blanks for: agent_id, orchestrator_url, LLM backend
- [] build_orchestrator_pack.py: assembles orchestrator zip
- [] Setup scripts (Linux/Mac + Windows)
- [] Verification script: checks Python, Ollama, dependencies, connectivity
- [] Quick-start documentation

Dependency Graph



Phases 5 and 6 can run in parallel after Phase 4. Phases 9 and 10 can run in parallel after Phase 8.

Estimated Effort

Phase	Focus	Lines of Code (est.)
-------	-------	----------------------

0	Scaffold, config, world loading	~500
1	Accounting (~76 columns, regimes)	~1,200
2	Orchestrator (clamping, settlement	~1,000
3	Agent application (server, brain,	~900
4	Integration (mock full run)	~500
5	Memory (within-run, across-run, di	~700
6	Prompts (all types, all phases, sc	~1,200
7	LLM backends (Ollama, OpenRouter,	~600
8	Scoring, diagnostics, cross-run DB	~700
9	Dashboard (Streamlit, plots, drill	~1,200
10	Regime experiments	~300
11	Deployment packaging	~400
Total	~9,200	
Tests	~3,000 additional	

Key Risks and Mitigations

Risk	Mitigation
LLM equity valuations unreasonable	DCF framework in prompt + cross-checks + soft bound
Agents produce degenerate decisions	Fingerprints + analytical tools + dispersion diagno
Environment unrealistic markets	Multinomial logit baseline + bounded validation + f
Bank failure cascades crash simulation	Capital replacement + distress constraints + cooldo
Death spiral (repeated Q1 defaults)	Consecutive-failure cap + environment-decided entry
Prompts exceed context window	Agent self-summarization + token budget + priority
M&A creates accounting complexity	Extensive unit tests for goodwill, impairment, cons
Cross-run DB schema drift	Version field + migration script
Agent memory grows unbounded	Self-summarization + user truncation + archival
Different LLM backends produce inconsistent quality	Fallback to mock on failure + quality diagnostics
World doc changes break assumptions	World docs as input parameters, validated by config
Dashboard slow on large datasets	SQLite indexing + lazy loading + pagination

Technology Stack

Component	Choice	Notes
Language	Python 3.11+	
Agent server	FastAPI + uvicorn	Async, Pydantic validation
Agent database	SQLite	Zero-config, portable
Analysis	pandas, numpy, scipy	Standard stack
LLM (local)	Ollama	Multiple models
LLM (cloud, default)	OpenRouter: deepseek/deepseek-v3.2	~\$2.50/run (80Q), tested 6/6 Grade
LLM (cloud, budget)	OpenRouter: mistralai/mistral-nemo	~\$0.25/run, tested 6/6 Grade A
LLM (cloud, other)	OpenRouter, Anthropic, OpenAI APIs	Per-agent configurable
Config	YAML	Regimes and worlds as separate YAM
Data	CSV (Compustat, debrief, scores)	Append-only
Serialization	pickle (checkpoints)	Full state
Communication	HTTP + JSON	Universal
Dashboard	Streamlit	Pure Python, interactive
Charts	Plotly	Interactive, exportable
Testing	pytest	

Packaging

zip + setup scripts

No Docker

Directory Structure

```
llm_firm_lab/  
  shared/  
    schemas.py  
    data_types.py  
    compustat_columns.py  
    config.py  
    world_loader.py  
  orchestrator/  
    engine.py  
    accounting/  
      postings.py  
      statements.py  
      measurement.py  
      goodwill.py  
      tax.py  
    clamping.py  
    settlement.py  
    bankruptcy.py  
    entry.py  
    ma.py  
    institution_capital.py  
    information_filter.py  
    compustat_writer.py  
    debrief.py  
    scoring.py  
    validation.py  
    diagnostics.py  
    product_market.py  
    shocks.py  
    dossier_manager.py  
    gazette_manager.py  
    memory_dispatch.py  
    checkpoint.py  
    cli.py  
  agent/  
    server.py  
    brain.py  
    analyst.py  
    memory.py  
    context_manager.py  
    tools/  
      trend.py  
      comparison.py  
      portfolio.py  
      forecasting.py  
      credit.py
```

```

valuation.py
llm/
  base.py
  ollama.py
  openrouter.py
  anthropic_backend.py
  openai_compat.py
  mock.py
  json_repair.py
prompts/
  builder.py
  templates/
    environment/
    firm/
    investment_bank/
    commercial_bank/
    credit_fund/
  config.py
dashboard/
  app.py
  pages/
    run_monitor.py
    time_series.py
    quarter_drill.py
    cross_run.py
    aggregate.py
  components/
    charts.py
    data_loader.py
    formatters.py
config/
  default.yaml
worlds/
  default/
    01_world_overview.md
    02_product_science.md
    03_manufacturing.md
    04_market_demand.md
    05_regulatory_environment.md
    06_rd_pathways.md
    07_financial_benchmarks.md
    08_accounting_rules.md
regimes/
  baseline_info.yaml
  full_transparency_info.yaml
  minimal_info.yaml
  asymmetric_banks_info.yaml
  baseline_gaap.yaml
  rd_capitalization.yaml
  fair_value.yaml
  cash_basis.yaml

```

```
data/  
  compustat_q.csv  
  debrief.csv  
  scores.csv  
  run_summaries.csv  
outputs/  
  {run_id}/  
    compustat_q.csv  
    debrief.csv  
    scores.csv  
    statements/  
    dossiers/  
    gazettes/  
    checkpoints/  
    logs/  
    memory_archives/  
tests/  
  orchestrator/  
  agent/  
  integration/  
docs/  
  world/  
  architecture/  
archive/
```